EJB3 Transactions

Transaction Management Features

- Provides a consistent programming model across different transaction APIs (JTA, JDBC,
- Hibernate, JPA)
- Supports declarative transaction management
- Provides a simpler API for programmatic transaction management than JTA
- Rollback on exception, rollback rules can be defined
- Atomicity, Consistency, Isolation, Durability

Isolation Level Definitions

- READ_UNCOMMITTED - dirty reads, non-repeatable reads and phantom reads can occur
- READ_COMMITTED - dirty reads are prevented
- REPEATABLE_READ - dirty reads and non-repeatable reads are prevented
- SERIALIZABLE - dirty reads, non-repeatable reads and phantom reads are prevented

Transaction Propagation/Attributes

- REQUIRED - support a current transaction, create a new one if none exists - DEFAULT
- MANDATORY - support a current transaction, throw an exception if none exists
- NESTED - subtransaction can be rolled back, behave like PROPAGATION_REQUIRED else
- NEVER - execute non-transactionally, throw an exception if a transaction exists
- NOT_SUPPORTED - execute non-transactionally, suspend the current transaction if one exists
- REQUIRES_NEW -create a new transaction, suspend the current transaction if one exists
- SUPPORTS - support a current transaction, execute non-transactionally if none exists

Declarative vs. Programmatic Transaction Management

Declarative

```
@Transactional(rollbackFor=ApplicationException.class)
public void processData {
      ...
      if( problem ) {
            throw new ApplicationException("It went wrong") ; // Rolled
            back
```

```
	}
}
```

Programmatic

```
public void processData {
	EntityTransaction userTransaction =
	entityManager.getTransaction();

	try{
		userTransaction.begin();
		// If everthing goes well, make a commit here.
		userTransaction.commit();
	} catch(Exception exception) {
		// Exception has occurred, roll-back the
		transaction.
		userTransaction.rollback();
	}
}
```

If you want to rollback the transaction in this case, you have to add the @ApplicationException(rollback=true) to your exception. Another way is to wrap the checked exception inside a unchecked exception (e.g. RuntimeException). But this way is not really preferred, because if a RuntimeException is thrown, the container discards the bean instance and creates a new one.